CENTRO UNIVERSITÁRIO DOM HELDER
CURSO DE CIÊNCIA DA COMPUTAÇÃO

RPG DE CONSOLE: A TORRE DO DRAGÃO

Aplicação de Programação Orientada a Objetos e dos princípios SOLID em Java

BELO HORIZONTE
2026

RPG DE CONSOLE: A TORRE DO DRAGÃO

Aplicação de Programação Orientada a Objetos e dos princípios SOLID em Java

Trabalho prático apresentado à disciplina de Paradigmas de Programação do curso de Ciência da Computação do Centro Universitário Dom Helder, como requisito parcial de avaliação.

BELO HORIZONTE

2026

SUMÁRIO

1 INTRODUÇÃO 3

2 REQUISITOS E COMO FORAM ATENDIDOS 4

3 ARQUITETURA 5

3.1 Estrutura de arquivos 5

3.2 Fluxo do programa 5

3.3 Diagrama de classes 6

4 EXPLICAÇÃO DO CÓDIGO 8

4.1 Personagens: abstração, encapsulamento e herança 8

4.2 Itens e inventário 9

4.3 Combate 10

4.4 Interface de console 11

4.5 Montagem do jogo e injeção de dependências 12

4.6 Aplicação dos princípios SOLID 12

5 COMO EXECUTAR 14

6 TESTES REALIZADOS 15

6.1 Testes de execução 15

6.2 Testes automatizados das regras 15

6.3 Capturas de tela 16

7 LIMITAÇÕES E MELHORIAS FUTURAS 22

REFERÊNCIAS 23

1 INTRODUÇÃO

Este documento descreve o desenvolvimento de um jogo de RPG em modo console, feito como trabalho prático da disciplina de Paradigmas de Programação. O enunciado pede uma aplicação que simule batalhas de RPG e que demonstre, no código, o uso correto de classes, herança e composição, polimorfismo, encapsulamento e dos cinco princípios SOLID.

O programa, chamado *A Torre do Dragão*, foi escrito em Java. O jogador escolhe uma entre três classes de herói (Guerreiro, Mago ou Arqueiro) e enfrenta três batalhas por turnos: contra um Goblin, um Esqueleto e, por fim, o chefão Dragão Ancestral. Entre as batalhas há um acampamento em que é possível usar e equipar itens.

O objetivo deste documento é mostrar como cada requisito foi atendido, explicar a arquitetura e o código, orientar a execução e registrar os testes realizados. O código-fonte completo está disponível no seguinte repositório público do GitHub:

https://github.com/FernandoJose-ESDHC/rpg-console-poo

O texto está organizado da seguinte forma: a seção 2 relaciona os requisitos com a solução; a seção 3 apresenta a arquitetura e o diagrama de classes; a seção 4 explica o código e a aplicação do SOLID; a seção 5 mostra como executar; a seção 6 registra os testes; e a seção 7 discute limitações e melhorias.

2 REQUISITOS E COMO FORAM ATENDIDOS

O Quadro 1 relaciona cada requisito do enunciado (numerados de R1 a R10) com a forma como foi atendido e o local do código em que isso pode ser conferido. Os caminhos são relativos à pasta *src*.

Quadro 1 – Rastreabilidade dos requisitos

Requisito	**Como foi atendido**	**Onde está no código**
R1 – Personagens com Vida, Ataque, Defesa e habilidades	Três classes de herói herdam de *Heroi*; os atributos ficam em *Personagem*; cada herói implementa sua habilidade em *executarHabilidade()*.	personagens/Personagem, Heroi, Guerreiro, Mago, Arqueiro
R2 – Combate por turnos contra inimigos simples e chefão	*Batalha.executar()* alterna o turno do herói e do inimigo; Goblin e Esqueleto usam o ataque padrão; o *ChefeDragao* usa o Sopro de Fogo.	combate/Batalha; personagens/Inimigo, Goblin, Esqueleto, ChefeDragao
R3 – Itens e inventário	Interface *Item*; *PocaoDeCura*; *Equipamento* com *Arma* e *Armadura*; *Inventario*; inimigos deixam recompensas.	itens/*; ui/MenuDeInventario
R4 – Interface de console	Menus numerados, barra de vida em texto e validação de toda entrada numérica.	ui/Console, Formatador; jogo/Jogo
R5 – Abstração, encapsulamento, herança/interfaces e polimorfismo	Classes abstratas, atributos *private*, hierarquias de herança, 6 interfaces e métodos sobrescritos.	Seção 4 deste documento
R6 – Os 5 princípios SOLID	Cada princípio tem exemplos concretos no código (Quadro 2).	Seção 4.6 deste documento
R7 – Repositório público, código pronto para compilar	Repositório no GitHub; compila só com o *javac*, sem bibliotecas externas.	README.md; seção 5
R8 – Diagrama de classes UML	Diagrama com todas as classes, atributos, métodos e relacionamentos (Figura 1).	docs/diagrama-classes.png, .svg e .dot
R9 – README estruturado	Instruções de compilação e execução, seção de SOLID e capturas de tela.	README.md
R10 – Compreensão do código	Código comentado em português, explicando o que cada parte faz e por quê.	Todos os arquivos

Fonte: elaborado pelo autor (2026).

3 ARQUITETURA

3.1 Estrutura de arquivos

O código está dividido em cinco pacotes, cada um com um assunto: **personagens** (heróis, inimigos e as interfaces de combate), **itens** (itens e inventário), **combate** (regra de dano e turnos), **ui** (tudo o que envolve texto na tela) e **jogo** (fluxo da partida). São 27 arquivos: 21 classes e 6 interfaces.

Código 1 – Estrutura de pastas do código-fonte

```
src/
├── Main.java                  ponto de entrada (monta e injeta dependências)
├── personagens/
│   ├── Combatente.java        interface: o que todo combatente faz
│   ├── Curavel.java           interface: quem pode ser curado
│   ├── UsaHabilidade.java     interface: quem tem habilidade especial
│   ├── Personagem.java        classe abstrata base
│   ├── Heroi.java             classe abstrata dos heróis
│   ├── Guerreiro.java, Mago.java, Arqueiro.java
│   ├── Inimigo.java           classe abstrata dos inimigos
│   └── Goblin.java, Esqueleto.java, ChefeDragao.java
├── itens/
│   ├── Item.java              interface de todo item
│   ├── PocaoDeCura.java
│   ├── Equipamento.java       classe abstrata (arma/armadura)
│   ├── Arma.java, Armadura.java
│   └── Inventario.java
├── combate/
│   ├── CalculadoraDeDano.java interface da regra de dano
│   ├── CalculadoraPadrao.java implementação da regra
│   └── Batalha.java           controla os turnos
├── ui/
│   ├── EntradaSaida.java      interface de entrada e saída
│   ├── Console.java           implementação com System.out e Scanner
│   ├── MenuDeInventario.java
│   └── Formatador.java
└── jogo/
    └── Jogo.java              fluxo da partida
```

Fonte: elaborado pelo autor (2026), arquivo *src/*.

3.2 Fluxo do programa

O fluxo de execução segue os passos abaixo:

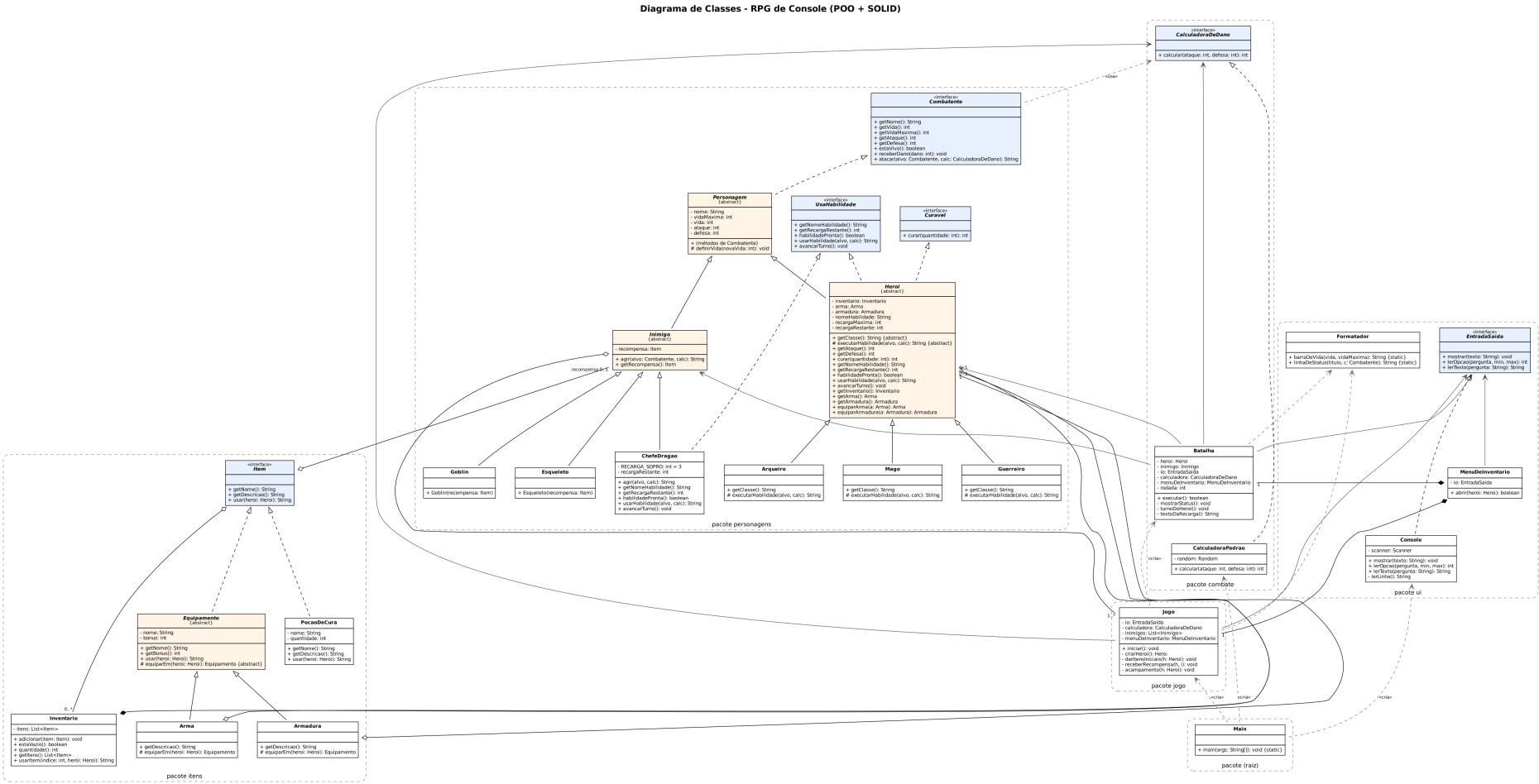
- o *Main* cria as dependências concretas (*Console*, *CalculadoraPadrao* e a lista de inimigos) e as entrega ao *Jogo*;
- *Jogo.iniciar()* pergunta o nome e a classe do herói e entrega os itens iniciais (duas poções e uma armadura);
- para cada inimigo da lista, é criada uma *Batalha*; *Batalha.executar()* repete as rodadas até um dos lados cair;
- em cada rodada, o jogador escolhe atacar, usar a habilidade ou usar um item; se o inimigo sobreviver, ele age;

- ao vencer, o herói recebe a recompensa do inimigo e passa pelo acampamento, onde pode equipar itens e ver o status;
- vencendo as três batalhas, o jogo mostra a vitória; se o herói cair, mostra o fim de jogo.

3.3 Diagrama de classes

A Figura 1, na página seguinte, apresenta o diagrama de classes UML completo. Classes com fundo laranja são abstratas e as de fundo azul são interfaces. As setas com triângulo vazio e linha contínua indicam herança; com linha tracejada, implementação de interface. O losango cheio indica composição, o losango vazio indica agregação e as setas tracejadas indicam dependência. Uma versão vetorial, que pode ser ampliada sem perder qualidade, está em *docs/diagrama-classes.svg* no repositório.

Figura 1 – Diagrama de classes UML do sistema



Fonte: elaborado pelo autor (2026).

4 EXPLICAÇÃO DO CÓDIGO

Esta seção explica as partes principais do código. Os trechos foram copiados diretamente dos arquivos-fonte.

4.1 Personagens: abstração, encapsulamento e herança

A classe abstrata *Personagem* reúne o que heróis e inimigos têm em comum: nome, vida, ataque e defesa. Ela é abstrata porque não existe um personagem genérico no jogo; só se criam subclasses concretas. Todos os atributos são privados e a vida só muda pelo método protegido *definirVida()*, que garante que o valor fique sempre entre zero e a vida máxima (Código 2). Isso é encapsulamento: nenhuma outra classe consegue deixar a vida negativa ou acima do máximo.

Código 2 – Encapsulamento da vida em Personagem

```
protected void definirVida(int novaVida) {
    if (novaVida < 0) {
        novaVida = 0;
    }
    if (novaVida > vidaMaxima) {
        novaVida = vidaMaxima;
    }
    this.vida = novaVida;
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/personagens/Personagem.java*.

A classe *Heroi* herda de *Personagem* e implementa as interfaces *Curavel* e *UsaHabilidade*. Ela define a regra comum a todas as habilidades (só usar se estiver pronta e, depois, entrar em recarga), mas deixa o efeito em si para cada subclasse, no método abstrato *executarHabilidade()* (Código 3). Assim, o Guerreiro dobra o ataque, o Mago ignora a defesa e o Arqueiro ataca duas vezes: o mesmo método chamado produz comportamentos diferentes (polimorfismo).

Código 3 – Regra comum das habilidades em Heroi

```
@Override
public String usarHabilidade(Combatente alvo, CalculadoraDeDano calculadora) {
    if (!habilidadePronta()) {
        return nomeHabilidade + " ainda está em recarga (" + recargaRestante + "
                rodada(s)).";
    }
    recargaRestante = recargaMaxima;
    return executarHabilidade(alvo, calculadora);
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/personagens/Heroi.java*.

Código 4 – Habilidade do Mago

```
@Override
protected String executarHabilidade(Combatente alvo, CalculadoraDeDano
        calculadora) {
    int ataqueMagico = getAtaque() * 2;                 // dobro do ataque
    int dano = calculadora.calcular(ataqueMagico, 0);   // defesa 0 = ignora a
            defesa
    alvo.receberDano(dano);
    return getNome() + " lança BOLA DE FOGO em " + alvo.getNome()
            + " e causa " + dano + " de dano (ignora a defesa)!";
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/personagens/Mago.java*.

O *Heroi* também sobrescreve *getAtaque()* e *getDefesa()* para somar os bônus da arma e da armadura equipadas. Os inimigos herdam de *Inimigo*, cujo método *agir()* faz um ataque normal. O *ChefeDragao* sobrescreve *agir()*: quando a recarga chega a zero, usa o Sopro de Fogo; antes disso, avisa o jogador (Código 5).

Código 5 – Comportamento do chefão

```
@Override
public String agir(Combatente alvo, CalculadoraDeDano calculadora) {
    if (habilidadePronta()) {
        return usarHabilidade(alvo, calculadora);
    }
    String resultado = atacar(alvo, calculadora);
    avancarTurno();
    // Avisa o jogador quando o sopro vai acontecer na próxima rodada.
    if (habilidadePronta()) {
        resultado += "\n>> O Dragão inspira profundamente... o SOPRO DE FOGO vem
                na próxima rodada!";
    }
    return resultado;
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/personagens/ChefeDragao.java*.

4.2 Itens e inventário

Todo item implementa a interface *Item*, com o método *usar(Heroi)*. A *PocaoDeCura* chama *heroi.curar()*; a classe abstrata *Equipamento* equipa o item e, se havia outro no mesmo espaço, devolve o antigo ao inventário (Código 6). *Arma* e *Armadura* só dizem em qual espaço o item é equipado. O *Inventario* retira o item da lista e aplica o efeito; sua lista é devolvida em modo somente leitura, para que nenhuma outra classe a altere diretamente.

Código 6 – Uso de um equipamento

```
@Override
public String usar(Heroi heroi) {
    Equipamento anterior = equiparEm(heroi);
    String mensagem = heroi.getNome() + " equipa " + nome + ".";
    if (anterior != null) {
        heroi.getInventario().adicionar(anterior);
        mensagem += " (" + anterior.getNome() + " voltou para o inventário)";
    }
    return mensagem;
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/itens/Equipamento.java*.

4.3 Combate

A *CalculadoraPadrao* implementa a regra de dano: o dano base é o ataque menos metade da defesa, com uma variação aleatória entre 80% e 120%, e nunca é menor que 1 (Código 7). O objeto *Random* é recebido pelo construtor; por isso, ao informar uma semente na execução (*java -cp out Main 42*), a partida se repete igual, o que facilita os testes.

Código 7 – Regra de dano

```
@Override
public int calcular(int ataque, int defesa) {
    int danoBase = ataque - defesa / 2;
    double variacao = 0.8 + random.nextDouble() * 0.4; // número entre 0.8 e 1.2
    int dano = (int) Math.round(danoBase * variacao);
    return Math.max(1, dano);
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/combate/CalculadoraPadrao.java*.

A classe *Batalha* é o coração do jogo (Código 8). O laço repete as rodadas enquanto os dois lados estiverem vivos. A *Batalha* não sabe qual inimigo está enfrentando: ela só chama *inimigo.agir()*, e o próprio objeto decide o que fazer. Também não calcula dano nem usa *System.out*; essas tarefas ficam com a calculadora e com a interface de entrada e saída, recebidas pelo construtor.

Código 8 – Laço principal da batalha

```
public boolean executar() {
    io.mostrar("\n========================================");
    io.mostrar("  BATALHA: " + heroi.getNome() + " x " + inimigo.getNome());
    io.mostrar("========================================");

    while (heroi.estaVivo() && inimigo.estaVivo()) {
        mostrarStatus();
        turnoDoHeroi();

        // O inimigo só age se ainda estiver vivo depois do ataque do herói.
        if (inimigo.estaVivo()) {
            io.mostrar(inimigo.agir(heroi, calculadora));
        }

        heroi.avancarTurno(); // diminui a recarga da habilidade
        rodada++;
    }

    if (heroi.estaVivo()) {
        io.mostrar("\n*** " + inimigo.getNome() + " foi derrotado! ***");
        return true;
    }
    io.mostrar("\n*** " + heroi.getNome() + " caiu em combate... ***");
    return false;
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/combate/Batalha.java*.

4.4 Interface de console

Toda a comunicação com o jogador passa pela interface *EntradaSaida*. A classe *Console* é a única que usa *System.out* e *Scanner*. O método *lerOpcao()* repete a pergunta até receber um número dentro do intervalo permitido, de modo que letras ou números inválidos não interrompem o jogo (Código 9). A classe *Formatador* monta a barra de vida e a linha de status, e a *MenuDeInventario* mostra os itens e pergunta qual usar.

Código 9 – Leitura de opção com validação

```
@Override
public int lerOpcao(String pergunta, int min, int max) {
    // Repete até o jogador digitar um número válido dentro do intervalo.
    while (true) {
        System.out.print(pergunta + " ");
        String linha = lerLinha().trim();
        try {
            int numero = Integer.parseInt(linha);
            if (numero >= min && numero <= max) {
                return numero;
            }
        } catch (NumberFormatException e) {
            // Não era número: cai na mensagem abaixo e pergunta de novo.
        }
        System.out.println("Opção inválida. Digite um número de " + min + " a " +
                max + ".");
    }
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/ui/Console.java*.

4.5 Montagem do jogo e injeção de dependências

O *Main* é o único lugar que escolhe as classes concretas: cria o *Console*, a *CalculadoraPadrao* e a lista de inimigos, e entrega tudo ao *Jogo* pelo construtor (Código 10). Para mudar a campanha, por exemplo acrescentando um inimigo, basta alterar essa lista; a classe *Jogo* não muda.

Código 10 – Montagem das dependências no Main

```
public static void main(String[] args) {
    Random random = (args.length > 0) ? new Random(Long.parseLong(args[0])) : new
            Random();

    EntradaSaida io = new Console();
    CalculadoraDeDano calculadora = new CalculadoraPadrao(random);

    // A campanha: dois inimigos comuns e o chefão, cada um com sua recompensa.
    List<Inimigo> inimigos = List.of(
            new Goblin(new PocaoDeCura("Poção de Cura", 35)),
            new Esqueleto(new Arma("Espada de Aço", 6)),
            new ChefeDragao(null) // o chefão é a última batalha, não precisa
                    deixar item
    );

    Jogo jogo = new Jogo(io, calculadora, inimigos);
    jogo.iniciar();
}
```

Fonte: elaborado pelo autor (2026), arquivo *src/Main.java*.

4.6 Aplicação dos princípios SOLID

Os princípios SOLID foram reunidos por Martin (2002) como orientações para um projeto orientado a objetos fácil de manter e de estender. O princípio da substituição tem origem no trabalho de Liskov e Wing (1994) sobre subtipos. O Quadro 2 mostra onde e como cada princípio aparece no projeto.

Quadro 2 – Aplicação dos princípios SOLID

Princípio	**Onde**	**Como foi aplicado**
S – Responsabilidade única	Batalha, CalculadoraPadrao, Inventario, MenuDeInventario, Console	Cada classe tem um só motivo para mudar: a Batalha só ordena os turnos; a calculadora só calcula o dano; o Inventario só guarda e usa itens; o MenuDeInventario só mostra o menu; o Console só lê e escreve no terminal.
O – Aberto/Fechado	Item, Inimigo, CalculadoraDeDano	Um novo item, inimigo ou regra de dano é adicionado criando uma classe nova. Batalha, Inventario e Jogo não precisam ser alterados, porque trabalham com as abstrações.
L – Substituição de Liskov	Heroi e Inimigo e suas subclasses	A Batalha recebe qualquer Heroi e qualquer Inimigo. O ChefeDragao sobrescreve agir() sem quebrar o contrato (recebe o alvo, causa dano e devolve um texto); o Heroi sobrescreve getAtaque() e continua devolvendo um valor válido.

Princípio	Onde	Como foi aplicado
I – Segregação de interfaces	Combatente, Curavel, UsaHabilidade	Em vez de uma interface grande, há três pequenas. Os inimigos comuns implementam só Combatente; não são obrigados a ter curar() ou usarHabilidade() sem uso.
D – Inversão de dependência	Jogo, Batalha, MenuDeInventario e Main	As classes de alto nível dependem das interfaces EntradaSaida e CalculadoraDeDano, recebidas pelo construtor. Só o Main conhece Console e CalculadoraPadrao. Isso permitiu testar as regras com uma calculadora de dano fixo (seção 6).

Fonte: elaborado pelo autor (2026).

Os conceitos de objeto, classe, herança, interface e pacote seguem a forma como são apresentados na documentação oficial da linguagem Java (ORACLE, 2026).

5 COMO EXECUTAR

É necessário ter o JDK 17 ou superior instalado (o projeto foi testado com o OpenJDK 21). Os mesmos comandos funcionam no Windows, no Linux e no macOS:

Código 11 – Comandos para baixar, compilar e executar

```
git clone https://github.com/FernandoJose-ESDHC/rpg-console-poo.git
cd rpg-console-poo
javac -encoding UTF-8 -d out -sourcepath src src/Main.java
java -cp out Main
```

Fonte: elaborado pelo autor (2026), arquivo *README.md*.

A opção *-sourcepath src* faz o compilador encontrar todas as classes usadas a partir do *Main*, e *-d out* coloca os arquivos compilados na pasta *out*. Se os acentos aparecerem errados no terminal do Windows, basta executar *chcp 65001* antes do jogo ou usar *java -Dstdout.encoding=UTF-8 -cp out Main*. Informar um número ao final (por exemplo, *java -cp out Main 42*) fixa a semente aleatória e repete sempre a mesma partida.

Para rodar os testes das regras, depois de compilar o jogo:

Código 12 – Comandos para executar os testes das regras

```
javac -encoding UTF-8 -cp out -d out testes/TesteRegras.java
java -cp out TesteRegras
```

Fonte: elaborado pelo autor (2026), arquivo *README.md*.

6 TESTES REALIZADOS

6.1 Testes de execução

O programa compilado foi executado em um terminal real, com as escolhas digitadas como faria um jogador. O Quadro 3 resume os testes e os resultados obtidos.

Quadro 3 – Testes de execução

Teste	**Procedimento**	**Resultado**
T1 – Compilação	javac com todas as verificações de aviso ativadas (-Xlint:all)	Compilou sem erros e sem avisos (27 classes).
T2 – Partida completa (Guerreiro)	Sementes 1 a 6, usando habilidades e itens	Vitória nas 3 batalhas em todas as 6 partidas.
T3 – Partidas com Mago e Arqueiro	Sementes 1 a 6 para cada classe	Vitória nas 3 batalhas em todas as 12 partidas.
T4 – Derrota	Mago, semente 1, apenas atacando	O herói cai contra o Dragão e aparece a mensagem de fim de jogo.
T5 – Entradas inválidas	Digitar abc, 0, 9, -1 e 7 nos menus	Todas recusadas; a pergunta se repete com a mensagem de opção inválida.
T6 – Habilidade em recarga	Escolher a habilidade logo após usá-la	Mensagem de aviso; o turno não é perdido e o menu reaparece.
T7 – Voltar do inventário	Abrir o inventário e escolher 0	O menu da mesma rodada reaparece e o inimigo não age.
T8 – Equipamentos	Equipar a Armadura de Couro e a Espada de Aço	Defesa do Guerreiro de 8 para 11 e ataque de 14 para 20.
T9 – Fim da entrada	Encerrar a entrada (Ctrl+D) no meio do jogo	O jogo encerra com a mensagem de despedida, sem erro.

Fonte: elaborado pelo autor (2026).

6.2 Testes automatizados das regras

O arquivo *testes/TesteRegras.java* verifica, sem bibliotecas externas, nove regras do jogo. Para que o resultado seja previsível, o teste usa uma calculadora de dano que sempre devolve 1, o que só é possível porque o jogo depende da interface *CalculadoraDeDano*. A saída obtida foi:

Código 13 – Saída da execução de TesteRegras

```
PASSOU  cura com vida cheia recupera 0
PASSOU  vida nunca fica negativa
PASSOU  cura não passa do máximo (70 + 35 -> 90)
PASSOU  arma soma no ataque (16 + 2 = 18)
PASSOU  trocar de arma devolve a antiga ao inventário
PASSOU  lista do inventário é somente leitura
PASSOU  habilidade entra em recarga após o uso
PASSOU  usar a habilidade em recarga não causa dano
PASSOU  Dragão usa o sopro nas rodadas 3 e 7 (nnSnnnS)

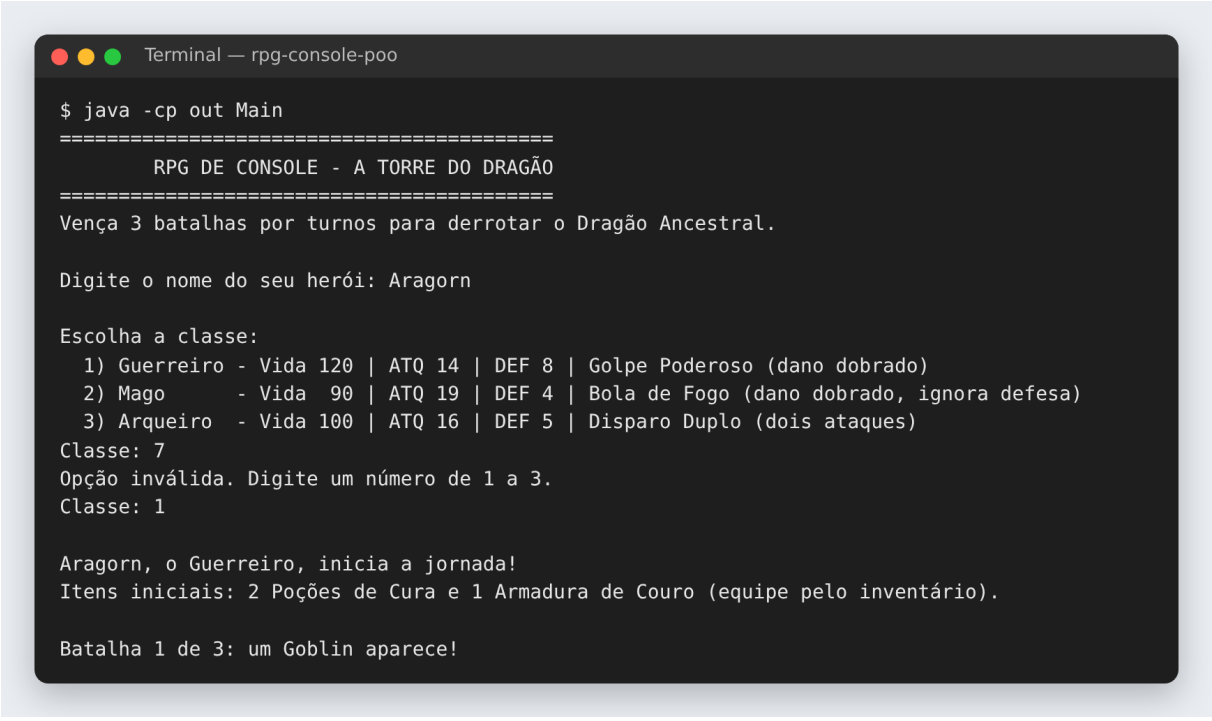
9 passaram, 0 falharam.
```

Fonte: elaborado pelo autor (2026), arquivo *testes/TesteRegras.java*.

6.3 Capturas de tela

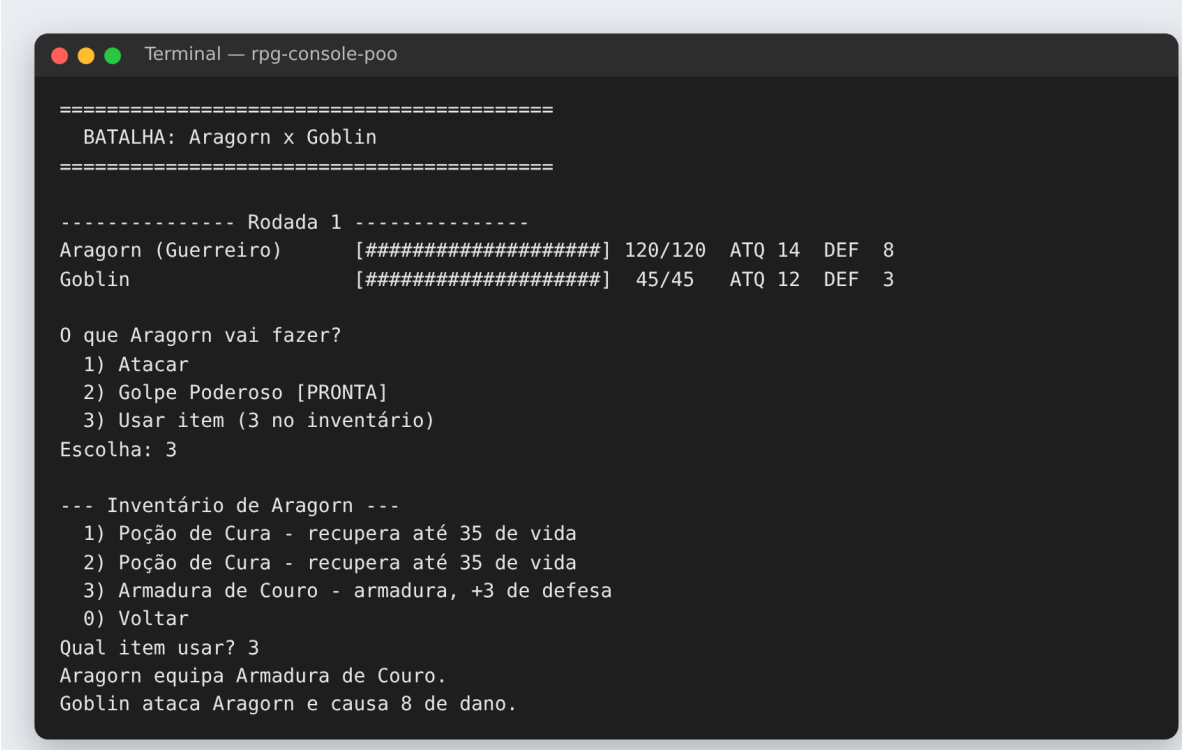
As figuras a seguir são trechos reais da saída do jogo, gravados durante as partidas dos testes T2 a T4 e convertidos em imagem.

Figura 2 – Início do jogo, escolha de classe e entrada inválida



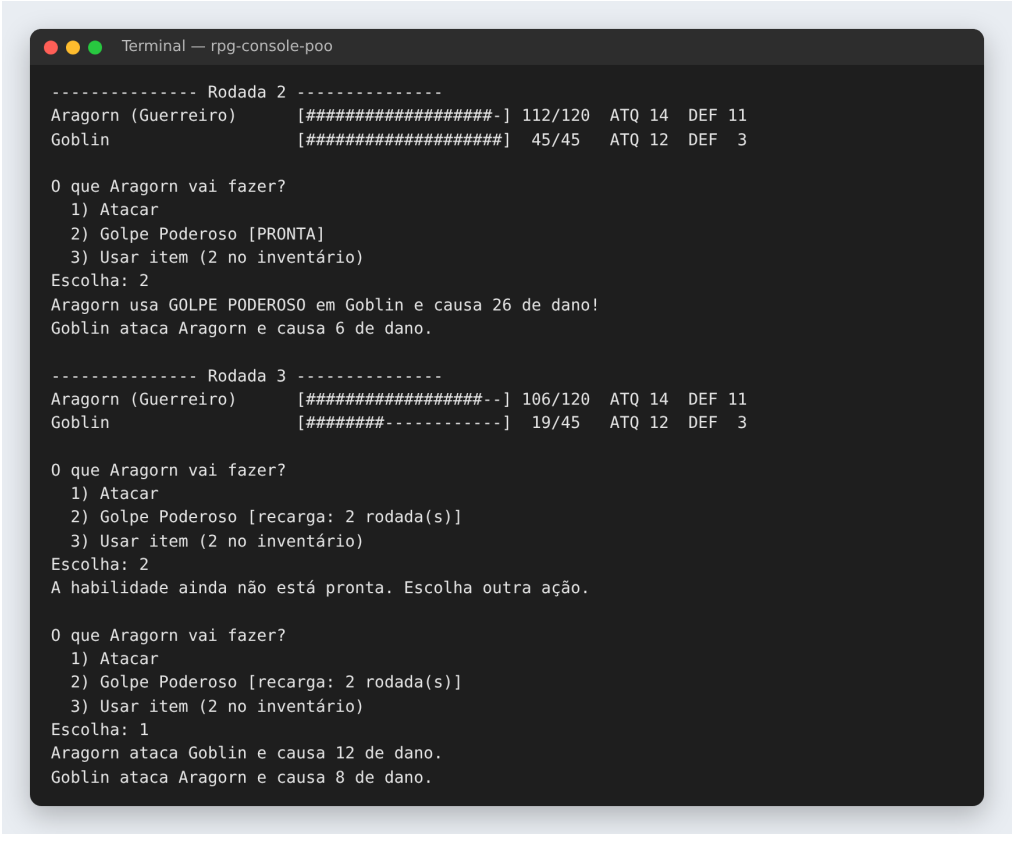
Fonte: elaborado pelo autor (2026).

Figura 3 – Primeira rodada: status, menu e uso do inventário



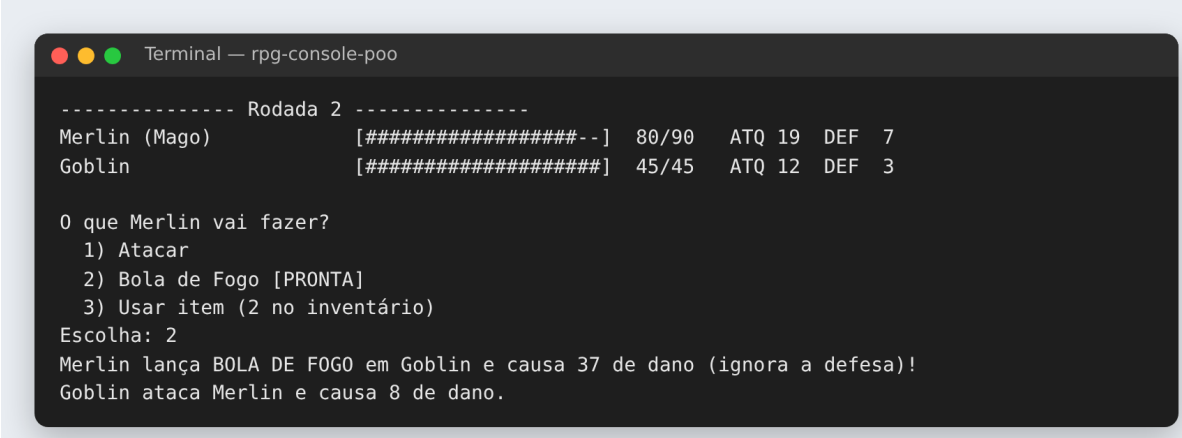
Fonte: elaborado pelo autor (2026).

Figura 4 – Golpe Poderoso e tentativa de uso durante a recarga



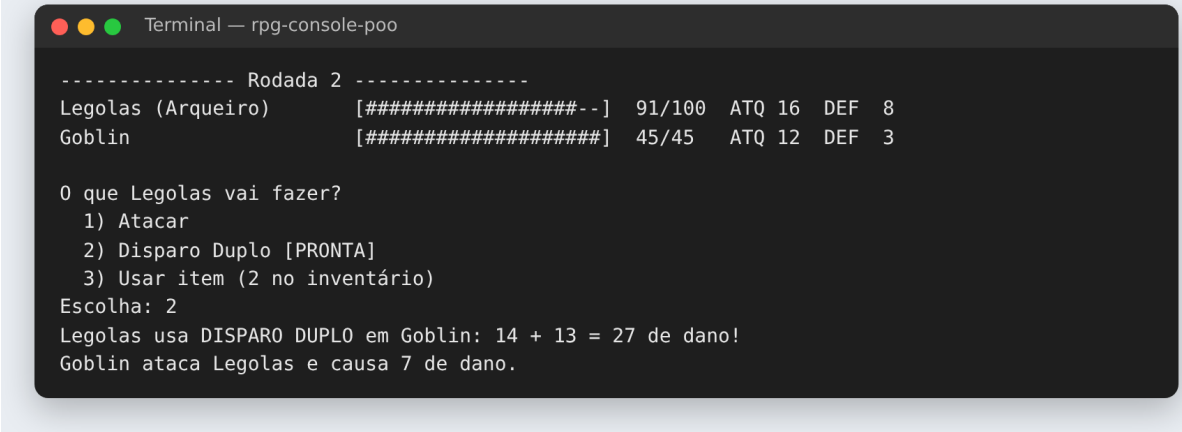
Fonte: elaborado pelo autor (2026).

Figura 5 – Bola de Fogo do Mago



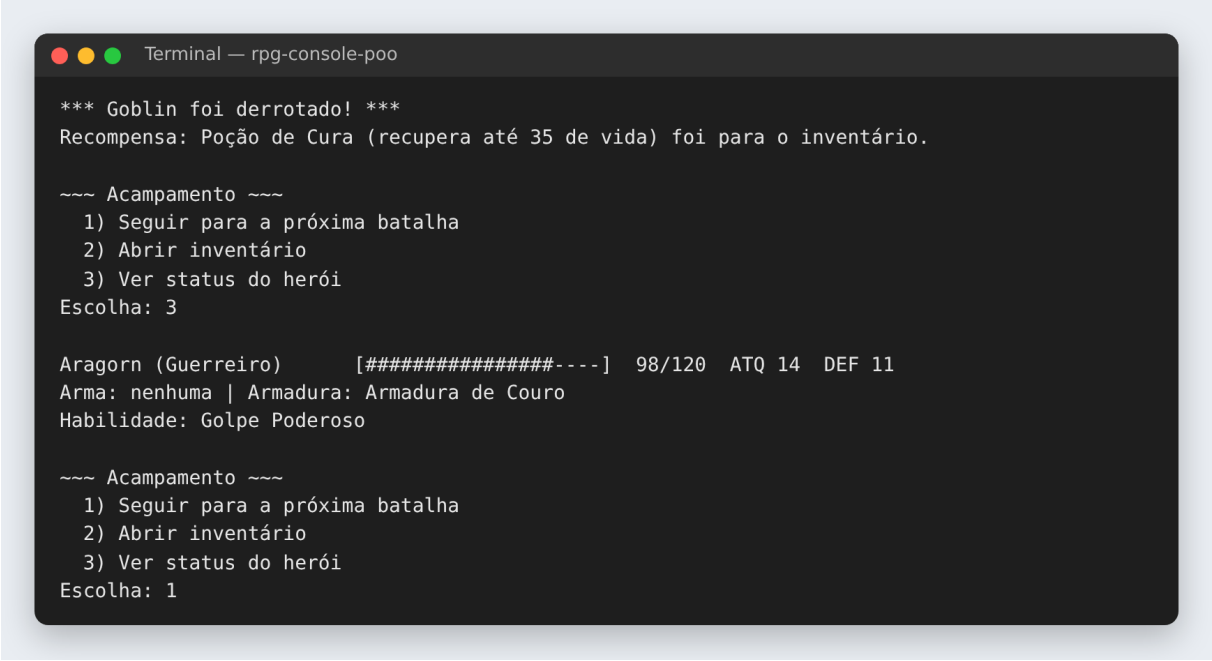
Fonte: elaborado pelo autor (2026).

Figura 6 – Disparo Duplo do Arqueiro



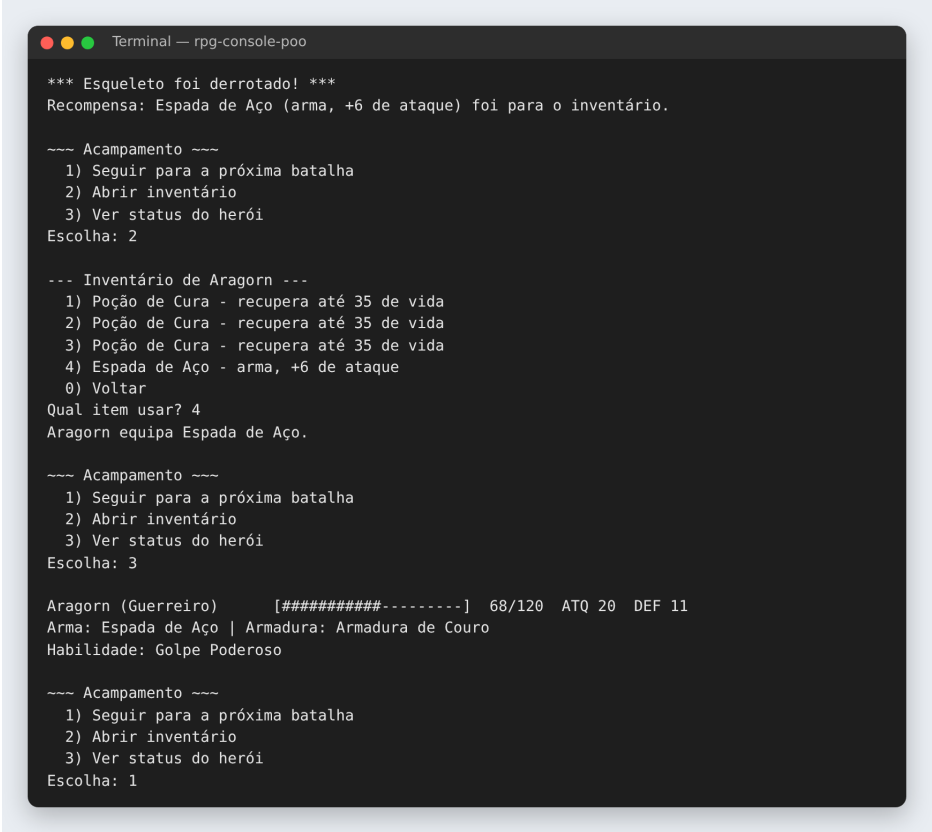
Fonte: elaborado pelo autor (2026).

Figura 7 – Recompensa e acampamento entre batalhas



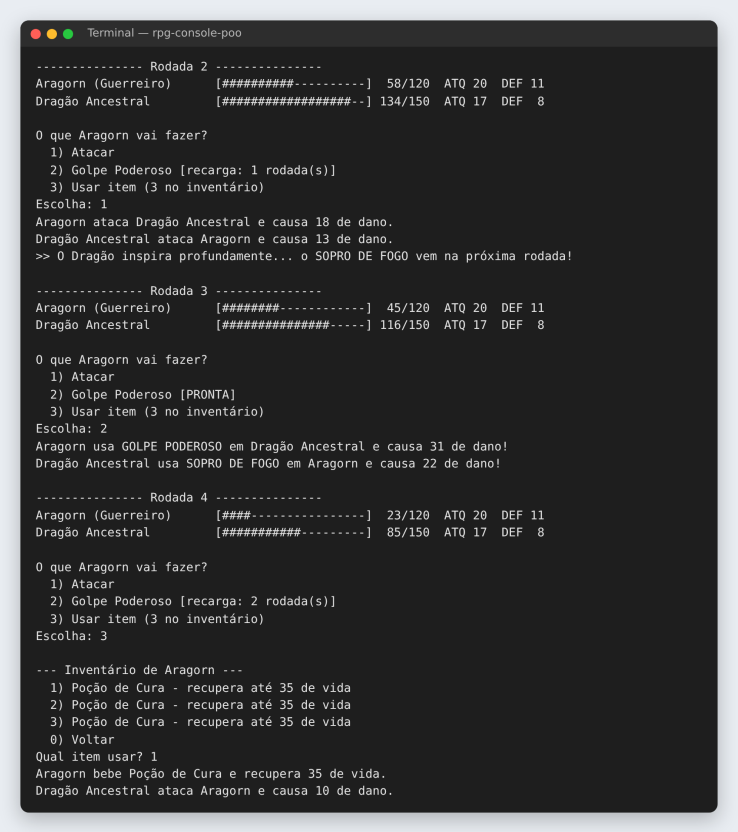
Fonte: elaborado pelo autor (2026).

Figura 8 – Equipando a Espada de Aço no acampamento



Fonte: elaborado pelo autor (2026).

Figura 9 – Batalha contra o Dragão: aviso, Sopro de Fogo e poção



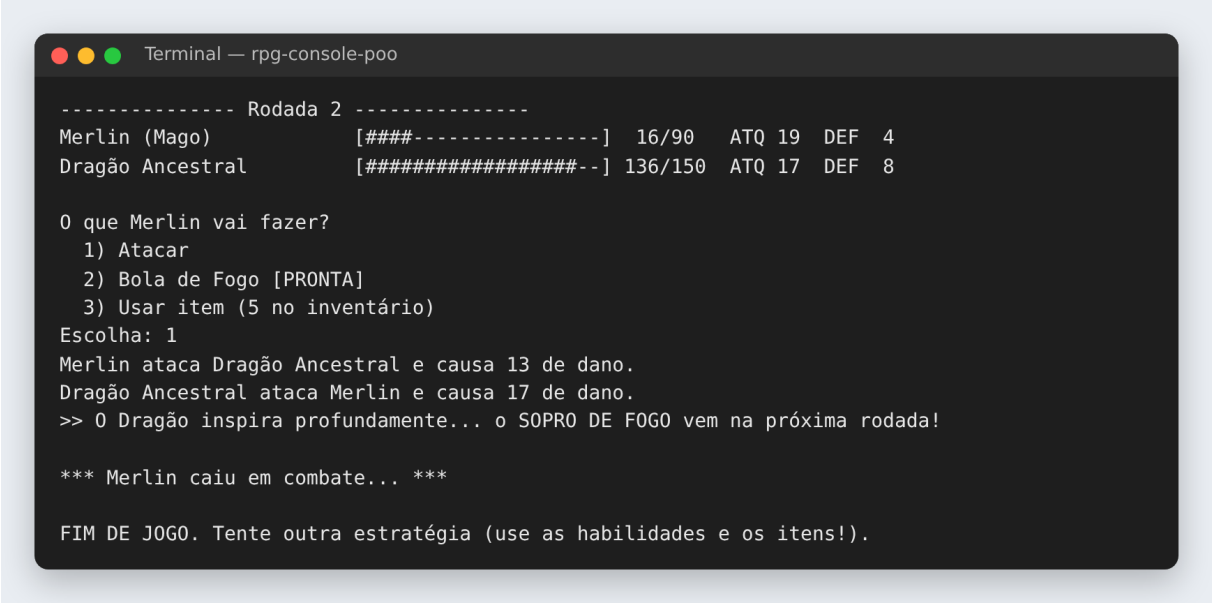
Fonte: elaborado pelo autor (2026).

Figura 10 – Vitória final



Fonte: elaborado pelo autor (2026).

Figura 11 – Derrota e fim de jogo



Fonte: elaborado pelo autor (2026).

7 LIMITAÇÕES E MELHORIAS FUTURAS

O trabalho tem as seguintes limitações conhecidas:

- o menu de escolha de classe, em *Jogo.criarHeroi()*, repete os atributos em texto e usa um *switch*; para incluir uma nova classe de herói, é preciso alterar esse método, além de criar a classe;
- a *Batalha* e o *Jogo* criam o *MenuDeInventario* com *new*, em vez de recebê-lo pelo construtor;
- a recarga da habilidade continua entre uma batalha e outra (se usada no fim de uma luta, a próxima começa com a habilidade em recarga);
- a partida não pode ser salva, e os testes de regras não usam um framework como o JUnit.

Como melhorias futuras, propõe-se: montar a lista de classes do menu a partir dos próprios objetos de herói; injetar o *MenuDeInventario*; adotar o JUnit para os testes; permitir salvar e carregar a partida; e acrescentar novos inimigos, itens e níveis de dificuldade, o que a arquitetura já facilita por seguir o princípio Aberto/Fechado.

REFERÊNCIAS

LISKOV, Barbara H.; WING, Jeannette M. A behavioral notion of subtyping. **ACM Transactions on Programming Languages and Systems**, v. 16, n. 6, p. 1811-1841, nov. 1994.

MARTIN, Robert C. **Agile software development**: principles, patterns, and practices. Upper Saddle River: Prentice Hall, 2002.

ORACLE. **Lesson**: object-oriented programming concepts. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/java/concepts/. Acesso em: 6 nov. 2026.